

คู่มือนักพัฒนา metang

เวอร์ชันที่ครอบคลุม: metang 0.1.0

เวอร์ชันเอกสาร: 1.0 ฉบับร่าง

วันที่: 2026-10-02

ตรวจสอบกับ: commit 403e36b (การเปลี่ยนแปลง outbox ตามหัวข้อ 8.2)

ฉบับภาษาไทยนี้แปลจากฉบับภาษาอังกฤษ (developer-guide.en.md) หากข้อความของสองฉบับต่างกัน ให้ถือฉบับภาษาอังกฤษเป็นหลัก

คู่มือนี้จัดทำสำหรับทีมพัฒนา metang คู่มือนี้อธิบายวิธีตั้งค่า แก้ไข ทดสอบ และจัดทำเอกสารของโค้ด คู่มือนี้ไม่ได้มาแทนเอกสารอื่น:

เอกสาร	ผู้อ่าน	ใช้สำหรับ
คู่มือฉบับนี้	ทีมพัฒนา	การทำงานกับโค้ด
AGENTS.md	ทีมพัฒนาและ coding agent	กฎแบบสั้น: รูปแบบโค้ด คำสั่ง และข้อความ commit
docs/documentation/maintenance-guide.en.md	เจ้าหน้าที่ไอทีของผู้ใช้งาน	การดูแลระบบหลังส่งมอบ (สำรองข้อมูล อัปเดตเฟิร์มแวร์) หัวข้อ 7 ของคู่มือนี้รวบรวมตัวแปรสภาพแวดล้อม (environment variable) ทุกตัว และค่าคงที่ทุกค่า (ขีดจำกัด ช่วงเวลา รูปแบบ) และหัวข้อ 10 รวบรวมข้อความแสดงข้อผิดพลาดทั้งหมด
docs/documentation/user-manual-staff.md, user-manual-student.md	ผู้ใช้งานปลายทาง	การใช้งานหน้าจอ
db/README.md, bruno/README.md	ทีมพัฒนา	คำสั่งของฐานข้อมูลและชุดทดสอบ API
docs/CMU-ENTRA-SSO.md, docs/Email_API_Manual.md	ทีมพัฒนา	ขั้นตอนการลงชื่อเข้าใช้ของ CMU และ CMU Email API

สารบัญ

- 1. ตั้งค่าเครื่องคอมพิวเตอร์
- 2. แผนผัง repository
- 3. คำสั่ง
- 4. ขั้นตอนการทำงานประจำวัน hook และ CI
- 5. การทดสอบ
- 6. ฐานข้อมูล
- 7. ข้อกำหนดการเขียนโค้ด
- 8. การแจ้งเตือนและงานเบื้องหลัง
- 9. จุดที่ควรระวังที่ทราบแล้ว
- 10. ดูแลเอกสารให้เป็นปัจจุบัน

1. ตั้งค่าเครื่องคอมพิวเตอร์

สิ่งที่ต้องมี:

- Node.js 24 (เวอร์ชันที่ระบุใน `.nvmrc`)
- Docker สำหรับ `npm run api:test` และ `npm run ci:local`
- Infisical CLI (ไม่บังคับ) หากไม่มี สคริปต์จะใช้ค่าใน `.env` บนเครื่อง
- สิทธิ์เข้าถึง environment `dev` ของ Infisical หรือฐานข้อมูล PostgreSQL ของตนเอง

ขั้นตอน:

1. ติดตั้ง dependency:

```
npm ci
```

คำสั่งนี้สร้าง Prisma Client ด้วย (`postinstall` สร้างไว้ใน `lib/generated/prisma` ซึ่ง Git ไม่ติดตาม) และเปิดใช้ Git hook ใน `.githubhooks/` (`prepare` ตั้งค่า `core.hooksPath`)

2. คัดลอก `.env.example` เป็น `.env` ตั้งค่า `INFISICAL_ENV=dev` ใส่ application ID ของ CMU, client secret และ `SESSION_SECRET` ที่ยาวอย่างน้อย 32 ตัวอักษรลงในไฟล์ หรือให้ Infisical โหลดค่าเหล่านี้ให้ หัวข้อ 7.1 ของคู่มือการดูแลรักษาระบบรวบรวมตัวแปรทุกตัว
3. ตรวจสอบว่า `DATABASE_URL` และ `DIRECT_URL` ชี้ไปยังฐานข้อมูลที่เป็นของตนเอง ห้ามชี้ไปยังฐานข้อมูล production จริงโดยเด็ดขาด
4. เริ่มเซิร์ฟเวอร์:

```
npm run dev
```

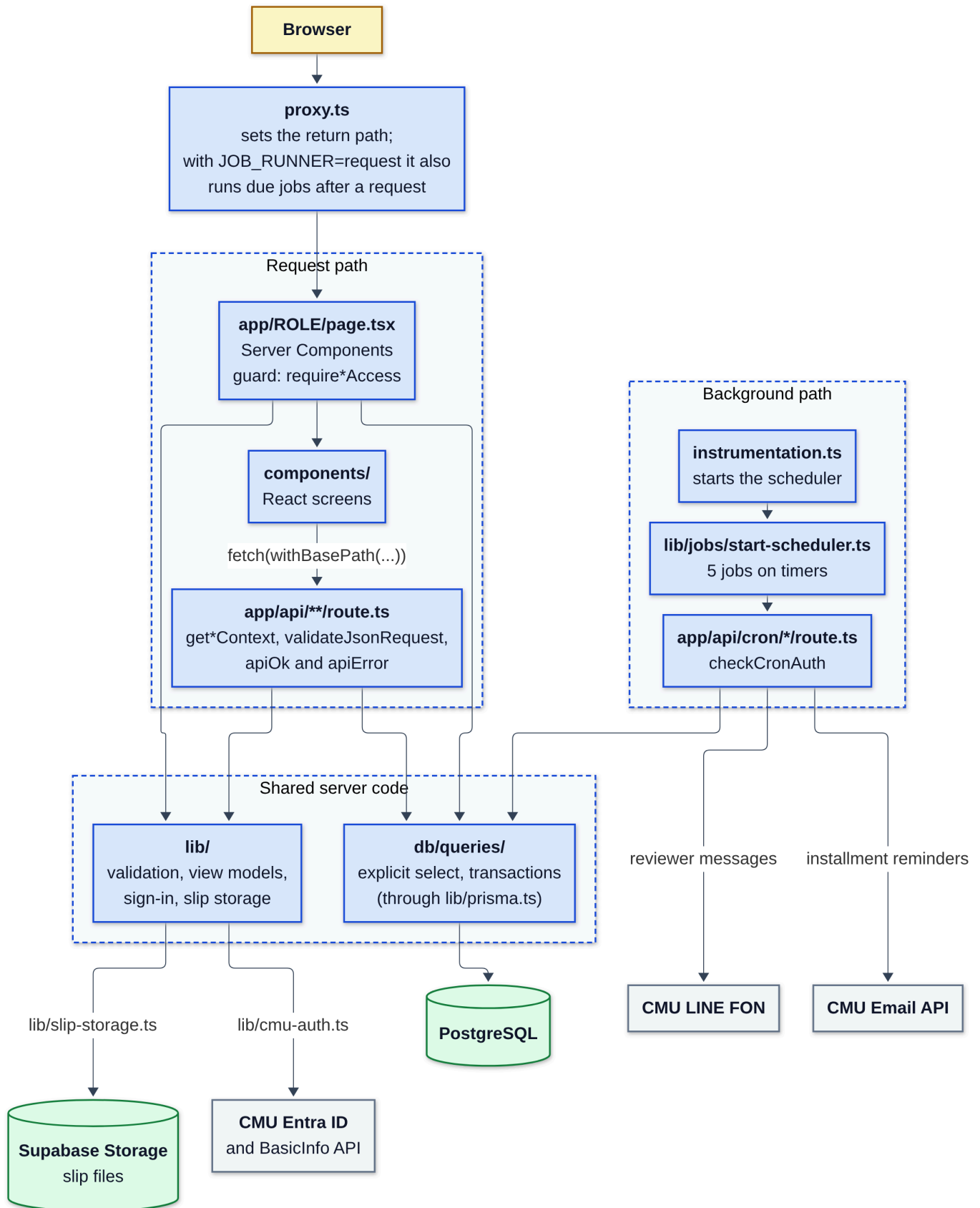
เซิร์ฟเวอร์รับการเชื่อมต่อที่พอร์ต 8080 เปิด <http://localhost:8080> แอปทำงานภายใต้ base path `/metang` (`PUBLIC_SUBPATH`) และ path ระดับราก (root) จะ redirect ไปยัง base path นั้น

5. ลงชื่อเข้าใช้ ทำอย่างใดอย่างหนึ่งต่อไปนี้ ลงทะเบียน callback URL ใน CMU Entra (`README.md`) หรือใช้ทางลัดสำหรับการพัฒนา คือ `DEV_API_BYPASS=true` ร่วมกับ `DEV_AS_ADVISOR`, `DEV_AS_ADMIN`, `DEV_AS_SUPERADMIN` หรือ `DEV_AS_EXECUTIVE` ตัวใดตัวหนึ่งที่ตั้งเป็น `true` (`lib/development-access.ts`) ทางลัดเหล่านี้ใช้ได้เฉพาะภายใต้ `next dev` หรือบน build ที่ตั้ง `DEBUG_MODE=true` หาก build ที่มีข้อมูลจริงตั้ง `DEBUG_MODE=true` บัญชี CMU ใดก็ลงชื่อเข้าใช้ได้ ดังนั้นให้ใช้เฉพาะกับเดโมที่ใช้ข้อมูลปลอม ไฟล์ `.env` สำหรับทางลัดของผู้ดูแลระบบ (Admin) มีลักษณะดังนี้:

```
INFISICAL_ENV=dev
DEV_API_BYPASS=true
DEV_AS_ADMIN=true
```

6. โหลดข้อมูลทดสอบหากต้องการ: `npm run db:seed` คำสั่ง `npm run db:reset` จะลบข้อมูลของแอปพลิเคชันก่อน ทั้งสองคำสั่งไม่ตรวจสอบว่าใช้ฐานข้อมูลใด ดังนั้นให้ตรวจสอบ `DATABASE_URL` ก่อน

2. แผนผัง repository



แผนภาพที่ 1. เส้นทางของคำขอและเส้นทางของงานเบื้องหลัง ที่มา: [images/developer-guide/code-map.mmd](#).

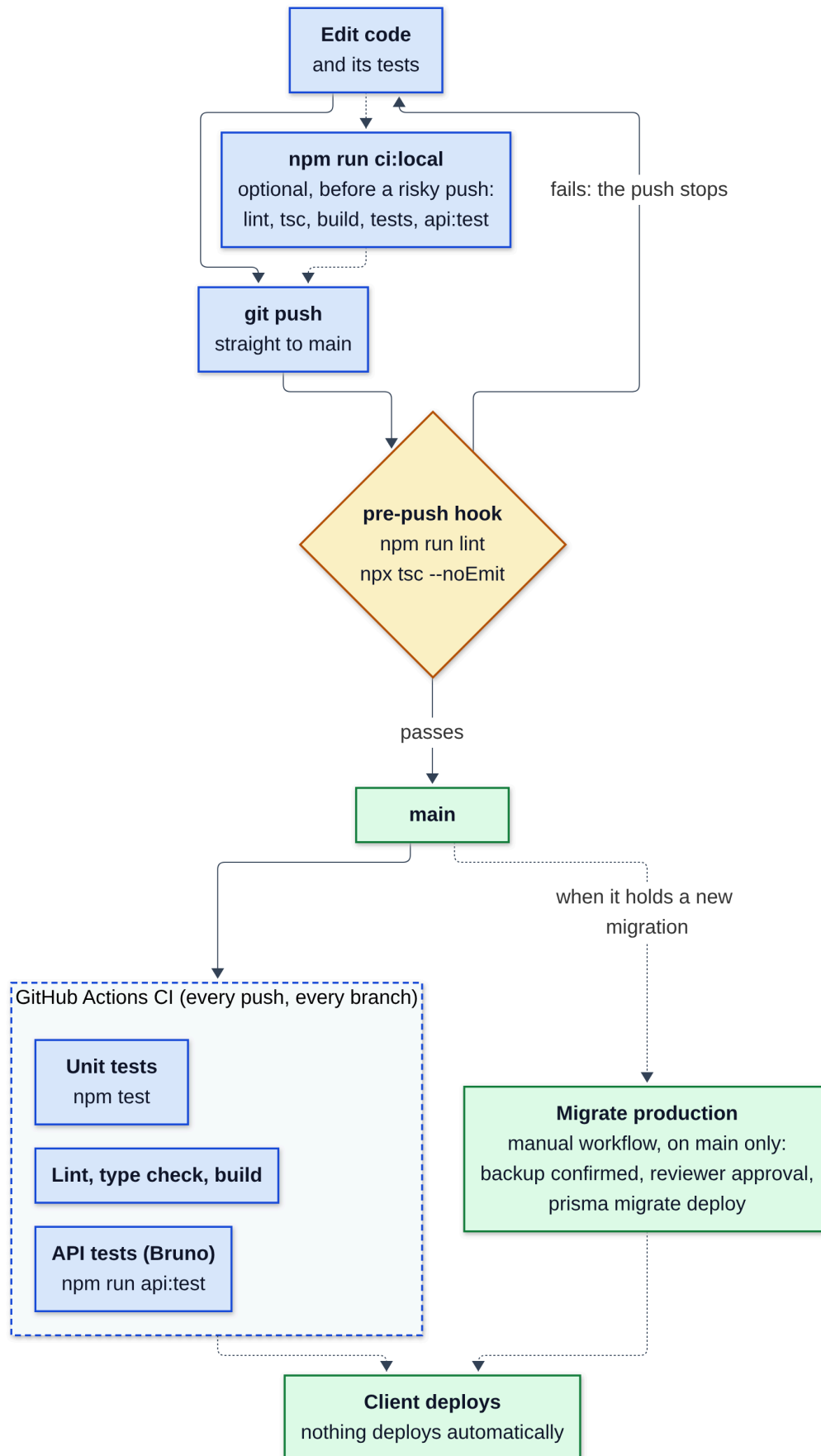
พาร	เนื้อหา
app/	Next.js App Router: หน้าของแต่ละบทบาท (student, advisor, admin, executive, superadmin), route handler ใน api/, login, หน้าทดสอบสำหรับนักพัฒนาใน demo/, api-docs (Swagger UI)
app/api/cron/	worker ของงานตามกำหนดเวลา (หัวข้อ 8)
components/	React component แยกตามบทบาท และ shared/ สำหรับส่วนที่ทุกบทบาทใช้ร่วมกัน (RoleShell, TopNav, SidebarNav)
hooks/	React hook ที่ใช้ร่วมกัน
lib/	logic ฟังก์ชันเวิร์กและ logic ที่ใช้ร่วมกัน: การลงชื่อเข้าใช้ (cmu-auth.ts, loan-auth.ts), การตรวจสอบความถูกต้องของข้อมูล, การแจ้งเตือน, งานตามกำหนดเวลา, การจัดเก็บสลิปโอนเงิน, view model
lib/generated/prisma/	Prisma Client (สร้างโดยอัตโนมัติ ไม่อยู่ใน Git)
db/schema.prisma	ต้นฉบับหลักของตารางและความสัมพันธ์
db/migrations/	migration แบบ SQL (26 รายการ ณ วันที่ 2026-10-02)
db/queries/	การอ่านและการเขียนฐานข้อมูลที่น่ากลับมาใช้ซ้ำได้ ให้อ้าง query ไว้ที่นี่ ไม่ใช่ใน route
db/seed.ts, db/clear.ts, db/simple-workflow.ts	สคริปต์ข้อมูลทดสอบและสคริปต์ล้างข้อมูล
db/schema.dbml, db/design/database_schema.pdf	แผนภาพของ schema (หัวข้อ 10)
tests/	unit test; tests/db/ เก็บ test ที่ต้องใช้ฐานข้อมูลจริง
bruno/	Bruno API collection (รันด้วย npm run api:test)
scripts/	with-infisical.mjs, api-test-isolated.mjs, ci-local.mjs, normalize-openapi.mjs, create-test-case-workbook.py
.githubhooks/, .github/workflows/	pre-push hook, CI workflow และ workflow ของ production migration
instrumentation.ts, proxy.ts	จุดเริ่มต้นของ scheduler สำหรับงานตามกำหนดเวลา; hook ของคำขอหน้าเว็บ (path ที่ใช้กลับมาหลังลงชื่อเข้าใช้, งานที่รับหลังคำขอ)
Dockerfile, deploy/	container image และตัวอย่างการตั้งค่า nginx (คู่มือการดูแลรักษาระบบ หัวข้อ 6.4)
public/openapi.json	คำอธิบาย API ที่สร้างขึ้นโดยอัตโนมัติ (หัวข้อ 7.3)
docs/	เอกสารทั้งหมด แผนภาพอยู่ใน docs/documentation/images/ (ต้นฉบับ Mermaid เป็น .mmd และภาพเป็น .png) docs/docs-pdf/ สร้างไฟล์ PDF ของคู่มือ (หัวข้อ 10)
supabase/config.toml	การตั้งค่า Supabase CLI ที่ปิด migration และ seed ไว้ เพราะ Prisma ดูแลทั้งสองอย่าง
test-cases/	workbook ของกรณีทดสอบ business logic

3. คำสั่ง

คำสั่ง	หน้าที่
<code>npm run dev</code>	รัน dev server ที่พอร์ต 8080 ผ่าน <code>scripts/with-infisical.mjs</code> คำสั่งจะหยุดทำงานหากไม่มี <code>.env</code> หรือ <code>INFISICAL_ENV</code>
<code>npm run dev-normal</code>	รัน dev server ที่พอร์ต 8080 โดยไม่ผ่านตัวห่อ (wrapper) ของ Infisical
<code>npm run build</code>	รัน <code>prisma generate</code> แล้วรัน <code>next build</code>
<code>npm run build:infisical, start:infisical</code>	build และเริ่มระบบด้วยค่าจาก Infisical (<code>start:infisical</code> ใช้พอร์ต 8080)
<code>npm run start</code>	รัน <code>next start</code> ที่พอร์ต <code>\$PORT</code> หรือพอร์ต 3000
<code>npm run lint</code>	ESLint (กฎ core web vitals ของ Next.js และกฎของ TypeScript)
<code>npm run tsc --noEmit</code>	ตรวจสอบ type
<code>npm test</code>	unit test ทั้งหมด (หัวข้อ 5)
<code>npm run api:test</code>	database test และ Bruno collection บนคอนเทนเนอร์ PostgreSQL 17 ชั่วคราว
<code>npm run ci:local</code>	การตรวจสอบของ CI ตามลำดับเดียวกับ CI แล้วล้างสิ่งที่สร้างไว้ (หัวข้อ 4.3)
<code>npm run openapi:generate</code>	สร้าง <code>public/openapi.json</code> ใหม่ (หัวข้อ 7.3)
<code>npm run db:generate</code>	สร้าง Prisma Client ใหม่
<code>npm run db:migrate</code>	สร้างและใช้ development migration (<code>prisma migrate dev</code>)
<code>npm run db:deploy, db:deploy:env</code>	ใช้ migration ที่ยังค้างอยู่ <code>db:deploy:env</code> ไม่ผ่าน Infisical
<code>npm run db:status</code>	แสดงสถานะของ migration
<code>npm run db:push, db:pull</code>	ส่ง schema ไปยังฐานข้อมูลโดยไม่สร้าง migration; อ่าน schema จากฐานข้อมูล
<code>npm run db:seed, db:reset</code>	โหลดข้อมูลทดสอบ; ลบข้อมูลของแอปพลิเคชันแล้วโหลดใหม่
<code>npm run db:studio</code>	Prisma Studio

คำสั่ง `db:*` ทั้งหมด ยกเว้น `db:generate` และ `db:deploy:env` รันผ่าน `scripts/with-infisical.mjs` สคริปต์นี้ โหลดตัวแปรของ environment ใน Infisical ที่ระบุไว้ใน `INFISICAL_ENV` หรือใช้ `.env` เมื่อไม่ได้ติดตั้ง CLI ไม่มี environment เริ่มต้น

4. ขั้นตอนการทำงานประจำวัน hook และ CI



แผนภาพที่ 2. สิ่งที่ต้องตรวจสอบการเปลี่ยนแปลง ไม่มีสิ่งใด deploy โดยอัตโนมัติ ที่นำ: [images/developer-guide/change-and-check.mmd](#).

4.1 กฎ

- ทีมพัฒนา push ตรงไปที่ `main` ไม่มี pull request ดังนั้น hook และ CI เป็นการตรวจสอบเพียงอย่างเดียว
- เขียน commit ในรูปแบบ `type(scope): summary` พร้อม body และ footer รูปแบบฉบับเต็มได้ใน `AGENTS.md`
- การเปลี่ยนแปลงที่แตะ schema ต้องมี migration และต้องมีหมายเหตุในข้อความ commit
- ห้ามรับ `git push --no-verify` UU `main`

ข้อความ commit มีลักษณะดังนี้ commit ที่ใช้ AI ช่วยต้องปิดท้ายด้วยบรรทัด `Co-Authored-By: ของ AI นั้นด้วย`

```
fix(db): enforce advisor assignment
```

Say what changed and why. Wrap the lines at about 76 characters.

No schema change and no migration.

Validation: npm run lint, npx tsc --noEmit, npm run build, npm test.

Not run: npm run api:test.

4.2 pre-push hook

`.github/hooks/pre-push` รับ `npm run lint` และ `npx tsc --noEmit` ก่อน push ทุกครั้ง hook นี้ตั้งใจให้ทำงานเร็ว จึงไม่ build และไม่รัน test

4.3 CI และ `npm run ci:local`

`.github/workflows/ci.yml` ทำงานทุกครั้งที่ push ไปยังทุก branch และสั่งรันเองได้ มี 3 job ดังนี้:

Job	ขั้นตอน
Lint, ตรวจสอบ type และ build	<code>npm ci</code> , <code>npm run lint</code> , <code>npx tsc --noEmit</code> , <code>npm run build</code> โดยใช้ <code>DATABASE_URL</code> และ <code>DIRECT_URL</code> ตัวอย่าง (placeholder)
Unit test	<code>npm ci</code> , <code>npm test</code>
API test (Bruno)	<code>npm ci</code> , <code>npm run api:test</code>

รับการตรวจสอบชุดเดียวกันบนเครื่องก่อน push การเปลี่ยนแปลงที่เสี่ยง:

```
npm run ci:local
```

คำสั่งนี้หยุดที่การตรวจสอบแรกที่ล้มเหลว เมื่อทุกขั้นตอนผ่าน บรรทัดสุดท้ายคือ `✓ all CI checks passed` เมื่อคำสั่งจบไม่ว่าจะผ่านหรือไม่ผ่าน คำสั่งจะลบผลลัพธ์ของ build ใน `.next` (ไม่รวม `.next/dev`) และคอนเทนเนอร์ `metang-test` เพิ่ม `-- --install` เพื่อรับ `npm ci` ก่อน ซึ่งจะลบ `node_modules`

การ build และการล้างสิ่งที่สร้างไว้เปลี่ยนแปลง `.next` ดังนั้นให้หยุดเซิร์ฟเวอร์ `next dev` และ `next start` ทุกตัวของโพลเดอร์นี้ก่อน คำสั่งจะไม่ยอมเริ่มทำงานหากมีโปรแกรมรอรับการเชื่อมต่อที่พอร์ต 8080, 3000 หรือ `$PORT` หรือหาก `.next/dev/lock` ระบุถึง dev server ที่ยังทำงานอยู่ คำสั่งตรวจไม่พบเซิร์ฟเวอร์ที่เริ่มเองด้วยมือบนพอร์ตอื่น

4.4 workflow ของ production migration

`.github/workflows/migrate-production.yml` รับ `prisma migrate deploy` กับ production ให้สั่งรันเองด้วยมือ และรับบน `main` เท่านั้น workflow นี้ต้องมี environment production ที่กำหนดผู้ตรวจสอบ (required reviewers)

และ secret `DIRECT_URL` ใน environment นั้น (ห้ามใช้ repository secret) workflow จะให้ยืนยันว่าสำรองข้อมูลแล้ว และให้พิมพ์ `migrate production migration` ไม่มี down script ดังนั้นข้อมูลสำรองเป็นทางเดียวที่ใช้ย้อนกลับได้ คู่มือการดูแลรักษาระบบ หัวข้อ 5.2 อธิบายการสำรองข้อมูล

5. การทดสอบ

`npm test` รัน `tests/*.test.ts` และ `tests/*.test.mjs` ด้วย `node:test` ผ่าน `tsx` โดยใช้ `--conditions=react-server` เพื่อให้ไฟล์ที่ import `server-only` โหลดได้ เมื่อวันที่ 2026-10-02 คำสั่งนี้รัน 584 test ใน 104 ไฟล์ รันที่ละไฟล์ได้ดังนี้:

```
npx tsx --conditions=react-server --test tests/fund-budget.test.ts
```

test มี 4 ประเภท ให้ทราบว่ากำลังอ่าน test ประเภทใด:

ประเภท	ตัวอย่าง	สิ่งที่พิสูจน์ได้
พฤติกรรม	<code>fund-budget.test.ts</code> , <code>installment-schedule.test.ts</code>	เรียกฟังก์ชันจริงด้วยข้อมูลนำเข้า แล้วตรวจสอบผลลัพธ์
ข้อความในซอร์ส	<code>payment-outcome-wiring.test.mjs</code> , <code>workflow.test.mjs</code>	อ่านไฟล์ซอร์สด้วย <code>readFileSync</code> แล้วตรวจสอบด้วย <code>assert.match</code> พิสูจน์ได้เพียงว่ามีโค้ดนั้นอยู่ ไม่ได้พิสูจน์ว่าโค้ดทำงานได้ การเปลี่ยนชื่อหรือการจัดรูปแบบใหม่อาจทำให้ test พัง
Migration	<code>*.migration.test.mjs</code> (11 ไฟล์)	อ่าน SQL ของ migration แล้วตรวจสอบข้อความของ constraint หรือ trigger
ฐานข้อมูล	<code>tests/db/*.test.ts</code>	รันกับฐานข้อมูล PostgreSQL จริง มีเพียง <code>npm run api:test</code> ที่รัน test เหล่านี้

`npm run api:test` (`scripts/api-test-isolated.mjs`) เริ่มคอนเทนเนอร์ `postgres:17` ชื่อ `metang-test` ที่พอร์ต 5433 ใช้ migration ทั้งหมด ใส่ข้อมูลทดสอบ รันไฟล์ `database test 3` ไฟล์ (47 test) เริ่มแอปที่พอร์ต 8081 และรัน Bruno collection สองรอบ ได้แก่ endpoint (41 request, 148 assertion) และการเดินตามลำดับของ `workflow/` (28 request, 62 assertion) หาก test ล้มเหลว คอนเทนเนอร์จะยังทำงานอยู่ ลบคอนเทนเนอร์ด้วย `docker rm -f metang-test` คำสั่งนี้ไม่ต้องใช้ `.env` และไม่แตะฐานข้อมูลจริง คำสั่งนี้ทดสอบการอัปเดตหรือดาวน์โหลดสลิป และ route ของการแจ้งเตือนไม่ได้ เพราะคอนเทนเนอร์ไม่มี credential ของ Supabase และไม่มีการ LINE หรืออีเมล `bruno/README.md` ระบุว่าอะไรรันได้และอะไรรันไม่ได้

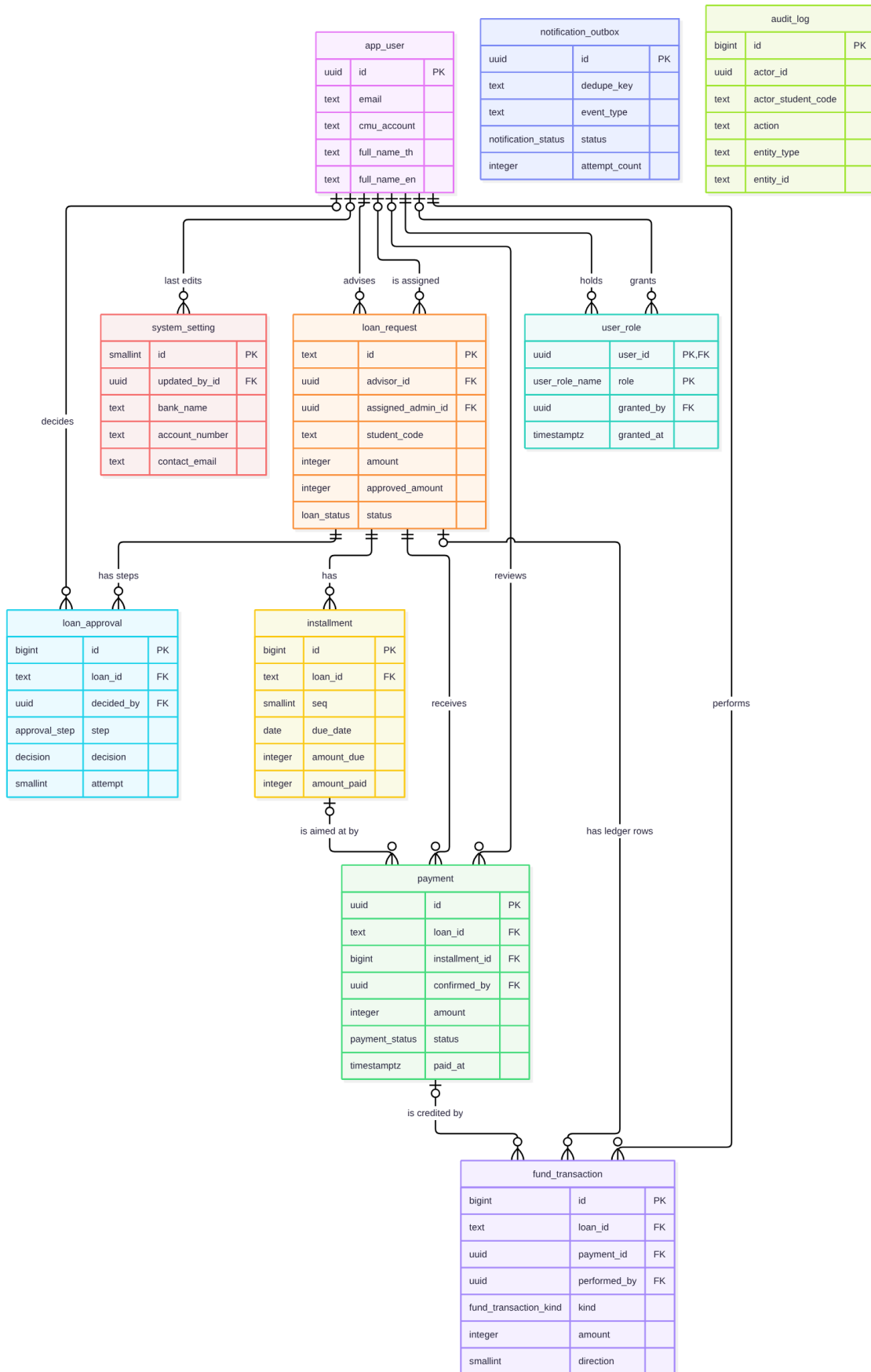
test ประเภทข้อความในซอร์สอ่านไฟล์แล้วจับคู่ข้อความของไฟล์นั้น (ตัดตอนจาก `tests/payment-outcome-wiring.test.mjs`):

```
const read = (file) => readFileSync(resolve(root, file), "utf8");
const worker = read("app/api/cron/deliver-payment-outcomes/route.ts");

test("the payment-outcome worker claims only its own event and sends by email, never FON", () => {
  assert.match(worker, /claimDueNotifications\((20, PAYMENT_OUTCOME_EVENT\)\/);
  assert.match(worker, /checkCronAuth\((request\)\/);
  assert.doesNotMatch(worker, /sendLineNotification|line-notification\/);
});
```

เมื่อเปลี่ยนพฤติกรรม ให้แก้ test ของพฤติกรรมนั้นใน commit เดียวกัน เมื่อเพิ่ม route ให้ตรวจ wiring test ของ route นั้น (เป็น test ประเภทข้อความในซอร์ส) และ Bruno collection

6. ฐานข้อมูล



แผนภาพที่ 3. ตารางและ foreign key ของตาราง แสดงเฉพาะคอลัมน์หลัก db/schema.prisma ระบุทุกคอลัมน์ และ db/schema.dbml เพิ่มหมายเหตุเกี่ยวกับกฎ ที่มา: [images/developer-guide/er-diagram.mmd](#).

6.1 เปลี่ยนแปลง schema

1. แก้ไข db/schema.prisma
2. รัน `npm run db:migrate` ตรวจสอบก่อนว่า DATABASE_URL และ DIRECT_URL ชี้ไปยังฐานข้อมูลของตนเอง
เพราะ `prisma migrate dev` อาจขอรีเซ็ตฐานข้อมูลเมื่อพบความแตกต่าง
3. อ่าน SQL ที่สร้างขึ้นใน db/migrations/<timestamp>_<name>/migration.sql ตรวจสอบ SQL ที่เขียนด้วยมือ
ด้วย BEGIN; และ COMMIT;
4. เพิ่มสิ่งที่ Prisma อธิบายไม่ได้ด้วยมือใน migration เดียวกัน ได้แก่ CHECK constraint, trigger, function และ view
ส่วน partial unique index เก็บไว้ใน schema ได้ (partialIndexes)
5. เพิ่มไฟล์ tests/<name>.migration.test.mjs สำหรับแต่ละกฎที่ฐานข้อมูลบังคับใช้
6. รัน `npm run api:test` คำสั่งนี้ใช้ migration ทุกตัวกับฐานข้อมูล PostgreSQL 17 ที่ว่างเปล่า
7. ตรวจสอบว่า schema และ migration ตรงกัน รันคำสั่งนี้กับฐานข้อมูลสำหรับพัฒนาหลังทำ migration:

```
npx prisma migrate diff --from-config-datasource --to-schema db/schema.prisma --exit-code
```

ผลที่ควรได้: No difference detected.

8. อัปเดตเอกสาร (หัวข้อ 10)

migration ที่เขียนด้วยมือเป็น transaction เดียว ตัวอย่างต่อไปนี้ตัดตอนจาก db/migrations/20261001160000_immutable_audit_log/migration.sql:

```
-- Say why the migration exists. Put the reasoning in comments.
BEGIN;

CREATE FUNCTION "public"."audit_log_block_mutation"() RETURNS trigger AS $$
BEGIN
  IF current_setting('methang.allow_audit_mutation', true) = 'on' THEN
    RETURN COALESCE(NEW, OLD);
  END IF;

  RAISE EXCEPTION 'audit_log is append-only; % is not allowed', TG_OP
    USING ERRCODE = 'check_violation';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER "audit_log_append_only"
  BEFORE UPDATE OR DELETE ON "public"."audit_log"
  FOR EACH ROW EXECUTE FUNCTION "public"."audit_log_block_mutation"();

COMMIT;
```

test *.migration.test.mjs ของ migration นี้อ่านข้อความ SQL (ตัดตอนแบบย่อจาก tests/admin-executive-loop.migration.test.mjs):

```
test("review-loop migration preflights legacy attempts before additive DDL", () => {
  const migration =
    read("db/migrations/20260904120000_admin_executive_review_loop/migration.sql");
  assert.match(migration, /^BEGIN;/);
  assert.match(migration, /ADD COLUMN "assigned_admin_id" UUID/);
  assert.match(migration, /COMMIT;\s*$/);
});
```

6.2 กฎที่ฐานข้อมูลบังคับใช้

- fund_transaction และ audit_log เป็นแบบ append-only trigger บล็อก UPDATE และ DELETE (และ TRUNCATE UU audit_log) มีเพียง db/seed.ts และ db/clear.ts ที่ปิด trigger เหล่านี้ โดยใช้ SET LOCAL methang.allow_fund_mutation = 'on' และ SET LOCAL methang.allow_audit_mutation = 'on' ภายใน transaction ห้ามทำเช่นนี้ในโค้ดของแอปพลิเคชันโดยเด็ดขาด

การปิด trigger มีผลเพียงหนึ่ง transaction (ตัดตอนจาก db/seed.ts):

```
async function wipeMockData(tx: Prisma.TransactionClient) {
  await tx.$executeRaw`SET LOCAL methang.allow_fund_mutation = 'on'`;
  // ... delete the mock rows
}
```

- fund_transaction_check_balance ปฏิเสธการ insert ที่ทำให้ยอดเงินกองทุนติดลบ
- audit_log_snapshot_actor ใส่ค่า audit_log.actor_name และ actor_role ก่อนการ insert ทุกครั้ง แอปพลิเคชันไม่ส่งค่าเหล่านี้
- loan_request อนุญาตให้นักศึกษาหนึ่งคนมีคำร้องที่ยังเปิดอยู่ได้หนึ่งคำร้อง (one_open_loan_per_student) และอนุญาตผู้บริหารหนึ่งคนใน user_role (one_executive_only) fund_transaction อนุญาตการเบิกจ่ายเงินหนึ่งครั้งต่อคำร้องหนึ่งคำร้อง (fund_transaction_one_disbursement_per_loan) และการชำระคืนหนึ่งรายการต่อรายการชำระเงินหนึ่งรายการ (fund_transaction_one_repayment_per_payment)
- CHECK constraint ได้แก่ audit_log_one_actor (ต้องมี actor_id หรือ actor_student_code อย่างใดอย่างหนึ่งเพียงตัวเดียว), fund_transaction_amount_positive, fund_transaction_direction_valid, fund_transaction_kind_direction_pairing, payment_amount_positive และ system_setting_singleton (มีหนึ่งแถว, id = 1) รูปแบบรหัสคำร้องขอกู้ยืม จำนวนงวดชำระ (1 ถึง 3) และ transfer_confirmed_at (ต้องไม่ก่อน disbursed_at) ก็มี CHECK constraint เช่นกัน
- next_loan_request_id() สร้างรหัสคำร้องขอกู้ยืม (REQYYYYMMDDNNNN, ใช้วันที่ตามเวลากรุงทพฯ) Prisma อธิบายฟังก์ชันนี้ไม่ได้ จึงมีอยู่เฉพาะใน SQL ของ migration
- หากต้องการดูกฎทุกข้อของตาราง ให้รัน \d <table> ใน psql หรืออ่านหมายเหตุใน db/schema.dbml และ SQL ใน db/migrations/ ไฟล์ db/schema.prisma ไม่มี CHECK constraint, trigger หรือ function

6.3 Query

วงการอ่านและการเขียนข้อมูลไว้ใน db/queries/ เช่น select อย่างชัดเจน ฟิลด์บัญชีธนาคารของคำร้อง (bankAccountNo, bankName, bankAccountName) ต้องส่งถึงเฉพาะผู้ดูแลระบบ (Admin) ผู้เบิกจ่ายเงินตามคำร้องเท่านั้น และ Bruno assertion ตรวจสอบข้อนี้ การตัดสินใจใช้ transaction ที่ระดับ isolation Serializable และเงื่อนไข updateMany ที่ตรวจสอบสถานะปัจจุบัน (decideAdminLoanRequest) เพื่อให้การตัดสินใจสองรายการที่เกิดพร้อมกันสำเร็จ

หนึ่งรายการและได้ 409 หนึ่งรายการ ให้ทำตามรูปแบบนี้เมื่อเพิ่มการตัดสินใจใหม่ ตัวอย่างต่อไปนี้ตัดตอนจาก `decideAdminLoanRequest` ใน `db/queries/loan-requests.ts` (`// ...` แถบบรรทัดที่ละไว้):

```
return prisma.$transaction(async (tx) => {
  const current = await tx.loanRequest.findFirst({
    where: { id, status: "pending_admin", OR: [{ assignedAdminId: null }, { assignedAdminId:
adminId }] },
    select: adminLoanDetailSelect,
  });
  if (!current) throw new AdminDecisionError("NOT_FOUND");
  // ... check the decision rules

  const changed = await tx.loanRequest.updateMany({
    where: { id, status: "pending_admin", OR: [{ assignedAdminId: null }, { assignedAdminId:
adminId }] },
    data: {
      status: nextStatus,
      approvedAmount: decision === "approved" ? approvedAmount : null,
      assignedAdminId: decision === "approved" ? adminId : null,
    },
  });
  if (changed.count !== 1) throw new AdminDecisionError("STALE_DECISION");

  // ... write the approval row and the audit row
  await enqueueReviewerNotifications(tx, { loanId: id, auditLogId: audit.id });
  // ...
}, { isolationLevel: Prisma.TransactionIsolationLevel.Serializable, timeout: 15000 });
```

หน้าของเจ้าหน้าที่เป็น Server Component ดังนั้นทุกอย่างที่ query ของหน้านั้นส่งกลับจะอยู่ใน HTML ของหน้า ซึ่งผู้ที่ลงชื่อเข้าใช้ด้วยบทบาทนั้นอ่านได้ทุกคน แต่ละหน้าเรียก query ของตนเองใน `db/queries/loan-requests.ts` (เช่น `getVerifySlipRequests`) และ query ตัดสินว่าหน้านั้นได้รับคำร้องและฟิลด์ใดบ้าง เมื่อเพิ่มฟิลด์ในหน้าจอของเจ้าหน้าที่ ให้เพิ่มฟิลด์นั้นใน query ของหน้านั้น อย่าขยาย query ให้ส่ง "ทุกอย่าง" เพียงเพื่อให้ฟิลด์ปรากฏ

ห้าม `import lib/prisma.ts` ใน Client Component โดยเด็ดขาด

7. ข้อกำหนดการเขียนโค้ด

7.1 รูปแบบโค้ด

เยื้อง 2 ช่องว่าง ใช้เครื่องหมายคำพูดคู่ ใส่เซมิโคลอน ใส่ trailing comma และไม่เกิน 100 ตัวอักษรต่อบรรทัด (`.prettierrc`) ตั้งชื่อ component แบบ PascalCase และตั้งชื่อตัวแปรแบบ camelCase ใช้ Server Component เป็นค่าเริ่มต้น เพิ่ม `"use client"` เฉพาะเมื่อ component ต้องใช้เบราว์เซอร์ ใช้ alias `@/` สำหรับการ import จากราก (root) ของ repository

โปรเจกต์นี้ใช้ Next.js 16 ซึ่งมี breaking change เมื่อเทียบกับเวอร์ชันเก่า ก่อนเขียนโค้ด Next.js ให้อ่านคู่มือที่ตรงกันใน `node_modules/next/dist/docs/` (`AGENTS.md` รวมเช่นเดียวกัน)

7.2 การลงชื่อเข้าใช้ บทบาท และ base path

- ขั้นตอนการลงชื่อเข้าใช้อยู่ใน docs/CMU-ENTRA-SSO.md (lib/cmu-auth.ts) session cookie ชื่อ cmu_session และมีอายุ 8 ชั่วโมง
- นักศึกษาไม่มีแถวใน app_user แต่ละ loan_request เก็บข้อมูลผู้กู้ไว้เอง (student_code พร้อมชื่อและอีเมลที่คัดลอกมา) เจ้าหน้าที่ที่มีแถวใน app_user และมีแถวใน user_role
- แต่ละหน้าป้องกันตนเองด้วย require*Access จาก lib/loan-auth.ts API route เรียกฟังก์ชัน get*Context() จากไฟล์เดียวกัน และต้องส่ง 401 หรือ 403 เองเมื่อได้ค่า null
- กฎของบทบาท (คู่มือการดูแลรักษาระบบ หัวข้อ 3.2): อาจารย์ที่ปรึกษาต้องไม่มีบทบาท admin หรือ super_admin ร่วมด้วย; มีผู้บริหารเพียงคนเดียวพอดี ผู้บริหารคนนี้มีบทบาท advisor ด้วย และแก้ไขที่แถวเดิม แอปพลิเคชันบังคับกฎข้อแรก ส่วนฐานข้อมูลบังคับเฉพาะ "ผู้บริหารหนึ่งคน"
- ผู้ที่ยังไม่ได้ลงชื่อเข้าใช้แล้วเปิดหน้าที่ถูกป้องกันจะถูกส่งไปที่ /login?next=<page> และกลับมาที่หน้านั้นหลังลงชื่อเข้าใช้ proxy.ts ส่งหน้านั้นให้ guard ผ่าน header และ lib/return-path.ts ปฏิเสธค่าที่ไม่ใช่หน้าของเว็บไซต์นี้
- ทุก fetch ฟังก์ชัน client และทุกฟังก์ชันที่เขียนด้วยมือต้องผ่าน withBasePath() จาก lib/base-path.ts หากไม่ใช้การเรียกจะใช้งานไม่ได้เมื่อ PUBLIC_SUBPATH ไม่ว่าง

ตัวอย่างหน้าที่ป้องกันตนเอง (ตัดตอนจาก app/admin/disburse-debt/page.tsx) guard จะ redirect ไปที่ /login หรือไปที่ /error?type=forbidden:

```
import { requireAdminAccess } from "@lib/loan-auth";

export default async function DisburseDebt() {
  await requireAdminAccess();
  // ... read with a function of db/queries, then render
}
```

API route ตอบ 401 เอง:

```
const context = await getStudentContext();
if (!context) return apiError("UNAUTHORIZED", "Authentication required", 401);
```

การเรียกจากฟังก์ชัน client (ตัดตอนจาก components/shared/disburse-debt/DisburseDebtCard.tsx):

```
import { withBasePath } from "@lib/base-path";

const res = await fetch(withBasePath(`/api/admin/loan-requests/${selectedRequest.id}/disburse`), {
  method: "POST",
  body: formData,
});
```

7.3 API route

- ตอบกลับด้วย apiOk(data) หรือ apiError(code, message, status) (lib/api-response.ts) body ของ response คือ { data } หรือ { error: { code, message } } พร้อม Cache-Control: no-store

- route ที่เปลี่ยนแปลงข้อมูลต้องเรียก `validateJsonRequest(request)` (ตรวจ same origin และ JSON) หรือ `isSameOrigin(request)` เมื่อไม่มี body (`lib/request-security.ts`)
- ตรวจ ID ด้วย `isLoanId` และส่งผลลัพธ์ผ่าน `serializeJson` (ฟังก์ชันนี้แปลง `Date` และ `bigint`)
- อธิบาย route ด้วย JSDoc tag เหนือ handler (`@tag`, `@body`, `@pathParams`, `@auth`, `@response`, `@add`) จากนั้นรัน `npm run openapi:generate` คำสั่งนี้รัน `next-openapi-gen` และ `scripts/normalize-openapi.mjs` แล้วเขียน `public/openapi.json` ให้ commit ผลลัพธ์ `/api-docs` แสดงไฟล์นี้ และ `/api/openapi` redirect ไปที่ไฟล์นั้น
- Bruno collection สร้างมาจากไฟล์นั้น อย่า import ไฟล์นั้นเข้ากับ collection เพราะจะเขียนกับทุก assertion ให้เพิ่มไฟล์ request ด้วยมือ (`bruno/README.md`)

route ที่ไม่มี body (ตัดตอนจาก `app/api/student/loan-requests/[id]/cancel/route.ts`):

```
/**
 * Cancel the current student's active loan request.
 * @tag Student loans
 * @pathParams LoanRequestIdParams
 * @auth cookieAuth
 * @response 200:LoanRequestDetailResponse
 * @add 401:ApiErrorResponse
 * @add 409:ApiErrorResponse
 */
export async function POST(request: Request, { params }: Params) {
  if (!isSameOrigin(request)) {
    return apiError("FORBIDDEN", "A same-origin request is required", 403);
  }
  const context = await getStudentContext();
  if (!context) return apiError("UNAUTHORIZED", "Authentication required", 401);

  const { id } = await params;
  if (!isLoanId(id)) return apiError("NOT_FOUND", "Loan request not found", 404);
  // ... a transaction with a guarded updateMany, then:
  return apiOk(serializeJson(loan));
}
```

route ที่มี JSON body เริ่มด้วย `validateJsonRequest` แล `isSameOrigin`:

```
export async function POST(request: Request) {
  const requestError = validateJsonRequest(request);
  if (requestError) return requestError;
  // ...
}
```

response ทั้งสองแบบมีหน้าตาดังนี้:

```
{ "data": { "id": "REQ202610020001", "status": "cancelled" } }
{ "error": { "code": "CONFLICT", "message": "The request can no longer be cancelled" } }
```

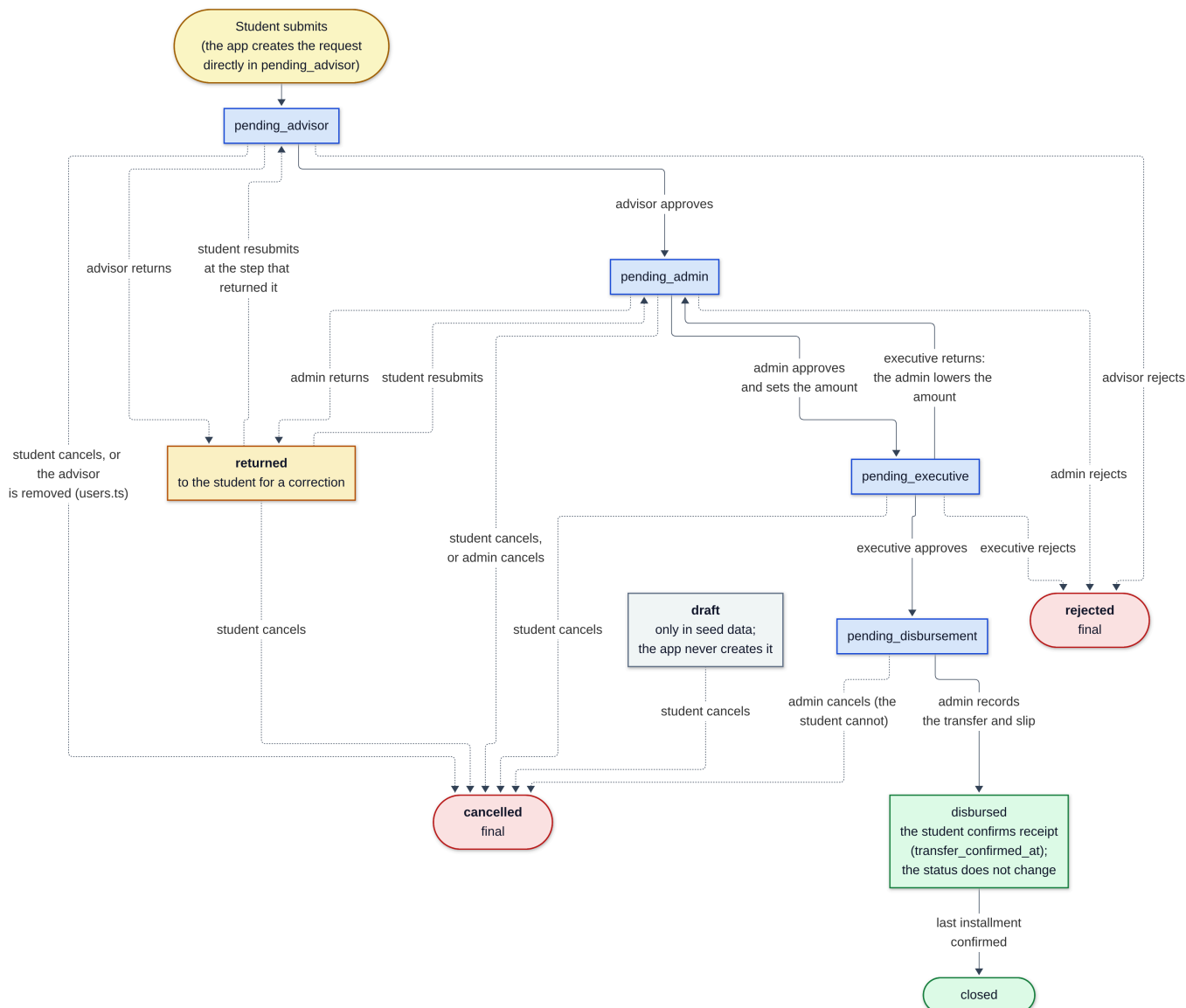
7.4 หน้าจอ

- หน้าจอของนักศึกษาที่มีข้อความทั้งภาษาไทยและภาษาอังกฤษผ่าน `app/student/StudentLanguageProvider.tsx` (`t(thai, english)` และ `map` สำหรับป้ายสถานะ) หน้าจอของเจ้าหน้าที่ใช้ภาษาไทยเท่านั้น ตัวอย่าง (ตัดตอนจาก `components/student/application/TempLoanApplicationPage.tsx`):

```
const { language, t } = useStudentLanguage();
// ...
{t("กลับหน้าหลัก", "Back to home")}
```

- ไฟล์บางไฟล์ที่มี `temp` หรือ `mock` ในชื่อยังใช้งานอยู่ ตัวอย่างเช่น component `Temp*` ใต้ `components/student/application/` คือแบบฟอร์มคำร้องขอกู้ยืมจริง และ `RequestsCard.tsx` ถึงเพดาน 500,000 บาทของยอดที่อนุมัติจาก `app/student/temp/tempMockData.ts` ค้นหา `import` ก่อนลบหรือ เปลี่ยนชื่อไฟล์เหล่านี้
- ไฟล์สลิปโอนเงินจะถูกเก็บใน bucket ส่วนตัวของ Supabase Storage (`lib/slip-storage.ts`, `SUPABASE_SLIP_BUCKET`) ขนาดสูงสุดคือ 1 MB (`MAX_SLIP_BYTES`) และ `lib/slip-file-type.ts` ตรวจสอบชนิดรูปภาพจริงจาก byte แรกของไฟล์ route `/api/payments/:id/slip` และ `/api/fund-transactions/:id/slip` ตรวจสอบสิทธิ์การเข้าถึงด้วย `lib/slip-access.ts`

7.5 ลำดับการเปลี่ยนสถานะของคำร้องขอกู้ยืม



แผนภาพ 4 การเปลี่ยนสถานะระหว่างค่าทั้ง 10 ค่าของ `loan_status` ที่มา: `images/developer-guide/loan-status-map.mmd`

- ฐานข้อมูลไม่ตรวจสอบลำดับการเปลี่ยนสถานะ แต่จะการดำเนินการจะอ่านสถานะปัจจุบันภายใน transaction แล้วอัปเดตโดยมี guard ที่สถานะนั้น (`updateMany` พร้อมสถานะที่คาดหวัง) ดังนั้นเมื่อมีสองการดำเนินการ พร้อมกัน จะสำเร็จเพียงหนึ่งรายการ และอีกรายการได้ 409
- แอปสร้างทุกคำร้องในสถานะ `pending_advisor` (`app/api/student/loan-requests/route.ts`) ค่า `draft` มีอยู่ใน enum และในข้อมูล seed รายการคำร้องของผู้พิจารณาไม่รวมสถานะนี้
- นักศึกษาขอยกเลิกคำร้องได้ในสถานะ `draft`, `returned`, `pending_advisor`, `pending_admin` หรือ `pending_executive` แต่ยกเลิกไม่ได้ใน `pending_disbursement` (`app/api/student/loan-requests/[id]/cancel/route.ts`) ผู้ดูแลระบบ (Admin) หรือผู้ดูแลระบบสูงสุด (SuperAdmin) ยกเลิกได้ในสถานะ `pending_admin` หรือ `pending_disbursement` (`cancelAdminLoanRequest`) การลบอาจารย์ที่ปรึกษาจะยกเลิกคำร้องของอาจารย์ท่านนั้นที่อยู่ในสถานะ `pending_advisor` เท่านั้น (`db/queries/users.ts`)
- หลังโอนเงินแล้ว นักศึกษายืนยันการรับเงิน ซึ่งจะบันทึกค่า `loan_request.transfer_confirmed_at` สถานะยังคงเป็น `disbursed` จนกว่าจะยืนยันงวดชำระสุดท้าย หน้าจอของนักศึกษาแสดงป้าย "กำลังชำระ" เฉพาะในเบราว์เซอร์ โดยดูจาก `isTransferConfirmed`

- นักศึกษามีคำร้องที่เปิดอยู่ได้ไม่เกินหนึ่งรายการ (one_open_loan_per_student) คำร้องที่เปิดอยู่ คือคำร้องที่มีสถานะอื่นนอกจาก `closed`, `rejected` หรือ `cancelled`

8. การแจ้งเตือนและงานเบื้องหลัง

คู่มือการดูแลรักษาระบบ หัวข้อ 2.3 ถึง 2.5 อธิบายเรื่องนี้จากมุมมองของผู้ปฏิบัติงาน

8.1 หลักการทำงาน

1. transaction ของฐานข้อมูลเขียนแถวลงใน `notification_outbox` ด้วย `enqueueNotification(tx, ...)` ค่า `dedupe_key` ต้องไม่ซ้ำกัน ดังนั้นเหตุการณ์เดียวกันจะไม่สร้างสองแถว
2. worker จะ claim แถวที่ถึงเวลาด้วย `claimDueNotifications(limit, eventType)` โดยใช้ `FOR UPDATE SKIP LOCKED` จึงรับได้หลาย server instance การ claim เป็น lease นาน 15 นาที แต่ละแถวส่งได้ 5 ครั้ง โดยมีเวลา 1, 5, 15 และ 60 นาที
3. worker ส่งข้อความและทำให้แถวเสร็จสิ้น (หัวข้อ 8.2) การรันหนึ่งรอบ claim ได้สูงสุด 20 แถว และส่งครั้งละ 5 แถว

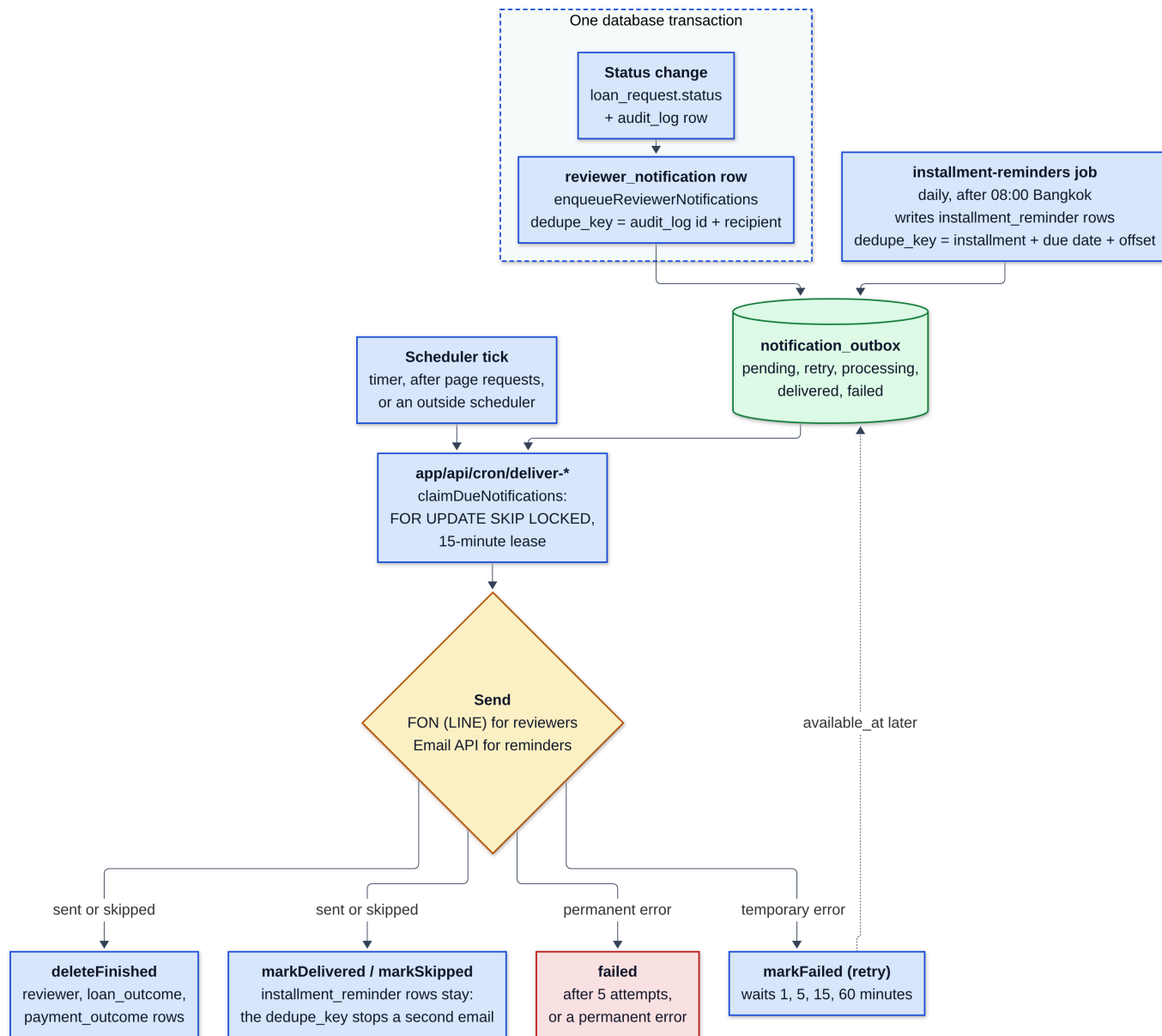
ขั้นตอนของผู้พิจารณาจะเขียนหนึ่งแถวต่อผู้รับหนึ่งคน ดังนั้นผู้ดูแลระบบ (Admin) สามคนจะได้สามแถว การแจ้งเตือนครบกำหนดชำระที่ไม่จำเป็นอีกต่อไป (งวดชำระถูกชำระแล้ว หรือเงินกู้ยืมยังไม่ได้เบิกจ่าย) จะถูกทำเครื่องหมายเป็น `delivered` โดย `delivered_at` ว่าง และใส่เหตุผลไว้ใน `last_error` job ที่ไม่ได้รันในวันใดวันหนึ่งจะไม่สร้างการแจ้งเตือนของวันนั้นย้อนหลัง

มี route สองตัวที่ส่งกันทีละไม่ใช้ outbox ได้แก่ `POST /api/notifications/fon` (การแจ้งเตือนทาง LINE ถึงผู้พิจารณาปัจจุบัน โดยมี cool-down 60 วินาทีในหน่วยความจำของแต่ละ instance) และ `POST /api/notifications/outlook` (อีเมลแจ้งเตือนครบกำหนดถึงนักศึกษา ไม่มีหน้าจอใด เรียกใช้)

ในโค้ด การ enqueue ทำงานภายใน transaction ของการเปลี่ยนแปลง และ worker claim ด้วย raw query (ตัดตอนจาก `db/queries/loan-requests.ts` และ `db/queries/notifications.ts`):

```
// inside prisma.$transaction(async (tx) => { ... })
await enqueueReviewerNotifications(tx, { loanId: id, auditLogId: audit.id });

// claimDueNotifications(limit, eventType)
const claimable = await tx.$queryRaw<{ id: string }[]>`
  SELECT id FROM notification_outbox
  WHERE event_type = ${eventType}
  AND status IN ('pending', 'retry', 'processing')
  AND available_at <= now()
  ORDER BY available_at
  LIMIT ${limit}
  FOR UPDATE SKIP LOCKED
`;
```



แผนภาพ 5 วงจรชีวิตของแถวใน outbox ที่มา: <images/developer-guide/notification-flow.mmd>

ประเภทเหตุการณ์	ช่องทาง	เขียนโดย
reviewer_notification	LINE ผ่าน FON API (lib/line-notification.ts)	enqueueReviewerNotifications (db/queries/notification-recipients.ts) ภายใน transaction ของการเปลี่ยนสถานะไปยังขั้นตอนของผู้พิจารณา (pending_advisor, pending_admin, pending_executive)
installment_reminder	อีเมลผ่าน CMU Email API (lib/email-api/)	job รายวัน installment-reminders
loan_outcome, payment_outcome	อีเมล	ไม่มีตั้งแต่ 2026-10-01 worker เพียงระบายแถวเก่าให้หมด

นักศึกษาได้รับอีเมลจากการแจ้งเตือนงวดชำระสองแบบนี้เท่านั้น อย่าเพิ่มอีเมลสำหรับการเปลี่ยนสถานะ โดยไม่ตกลงกับสำนักงานกองทุนก่อน

8.2 ทำให้แถวเสร็จสิ้น

- แถว `reviewer_notification`, `loan_outcome` หรือ `payment_outcome` ที่ส่งแล้วหรือข้ามแล้วจะถูกลบ ด้วย `deleteFinished(id)` คีย์ของแถวคือแถวใน audit log ของการเปลี่ยนสถานะหนึ่งครั้ง จึงไม่มี สิ่งใด enqueue แถวนั้นอีก
- แถว `installment_reminder` จะยังคงอยู่ และถูกทำเครื่องหมายด้วย `markDelivered(id)` หรือ `markSkipped` ค่า `dedupe_key` ของแถว (`installment-reminder:<installmentId>:<dueDate>:<offsetDays>`) คือสิ่งที่ป้องกันไม่ให้ scheduler ที่ restart ส่งอีเมลแจ้งเตือนเดียวกันซ้ำสองครั้ง ห้ามใช้ `deleteFinished` กับแถวประเภทนี้

```
// app/api/cron/deliver-fon/route.ts: a sent or skipped reviewer row is deleted
await deleteFinished(row.id);

// app/api/cron/deliver-reminders/route.ts: an installment reminder stays
await markDelivered(row.id);
await markSkipped(row.id, decision.reason);

// on an error: the first call fails the row for good, the second retries it after a wait
await markFailed(row.id, message, { permanent: true });
await markFailed(row.id, message, {});
```

8.3 งานตามกำหนดเวลา

`instrumentation.ts` เริ่ม `lib/jobs/start-scheduler.ts` เมื่อ server เริ่มทำงาน แต่ละ job เรียก route handler ของ `app/api/cron/<name>` โดยตรง ด้วย bearer token `CRON_SECRET` ที่ `checkCronAuth` ตรวจสอบ `JOB_RUNNER` เป็นตัวเลือก trigger ได้แก่ `timer` (ค่าเริ่มต้น), `request` (หลังจาก request ของหน้าเว็บ ผ่าน `proxy.ts` สำหรับ host ที่ freeze server ที่ไม่ได้ใช้งาน) หรือ `off` (scheduler ภายนอกเรียก route)

Job	ความถี่
<code>deliver-fon</code>	ทุก 1 นาที
<code>deliver-reminders</code> , <code>deliver-loan-outcomes</code> , <code>deliver-payment-outcomes</code>	ทุก 3 นาที
<code>installment-reminders</code>	วันละครั้ง ในการตรวจสอบรอบแรกหลัง 08:00 เวลากรุงเทพฯ job นี้เขียนหนึ่งแถวสำหรับทุกงวดชำระที่ยังไม่ได้ชำระ ซึ่งจะครบกำหนดในอีก 3, 1 หรือ 0 วัน หรือเลยกำหนดมาแล้ว 1, 3 หรือ 7 วัน

วิธีเพิ่ม job:

- เพิ่ม `app/api/cron/<name>/route.ts` และเรียก `checkCronAuth(request)` ก่อนเป็นอันดับแรก
- เพิ่ม job ลงใน `JOB` ใน `lib/jobs/start-scheduler.ts`
- เพิ่ม wiring test เหมือน `tests/payment-outcome-wiring.test.mjs`
- อัปเดตตัวอย่าง cron และตาราง job ในคู่มือการดูแลรักษาระบบ หัวข้อ 2.3 แล้วรัน `npm run openapi:generate`

route ของ job (ตัดตอนจาก `app/api/cron/deliver-fon/route.ts`):

```
import { checkCronAuth } from "@lib/notifications/cron-auth";

export const maxDuration = 60;
```

```

async function handle(request: Request) {
  const authError = checkCronAuth(request);
  if (authError) return authError;
  // ... claim, send, finish the rows
  return apiOk(serializeJson({ processed, delivered, skipped, failed }));
}

export const GET = handle;
export const POST = handle;

```

รายการของ job นี้ใน JOBS (lib/jobs/start-scheduler.ts):

```

{
  path: "/api/cron/deliver-fon",
  shouldRun: everyMinutes(1),
  load: () => import("@/app/api/cron/deliver-fon/route"),
},

```

9. จุดที่ควรระวังที่ทราบแล้ว

- **โฟลเดอร์ .next ใช้ร่วมกัน** npm run dev, npm run build, npm run start, npm run api:test และ npm run ci:local ใช้โฟลเดอร์นี้ร่วมกัน การ build หรือการล้างโฟลเดอร์ ขณะที่ server กำลังรันอยู่ จะทำให้ server นั้นล้มด้วยข้อผิดพลาด Cannot find module .next/server/middleware-manifest.json ให้หยุด server ก่อน
- **พอร์ต** โหมด dev ใช้พอร์ต 8080 ส่วน api:test ใช้พอร์ต 8081 สำหรับแอปและพอร์ต 5433 สำหรับ PostgreSQL dev server ที่รันอยู่จะทำให้ api:test รันไม่ได้ (คำสั่งนี้อ่าน .next/dev/lock)
- **tsconfig.json เปลี่ยนเอง** Next เขียนไฟล์นี้ใหม่ระหว่าง dev, build และ api:test ตรวจสอบไฟล์นี้ก่อน commit และถ้าคุณหากมีการเปลี่ยนแปลง:

```

git diff tsconfig.json
git show HEAD:tsconfig.json > tsconfig.json

```

- **Prisma Client ไม่อยู่ใน Git** หลังจาก pull การเปลี่ยนแปลง schema ให้รัน npm run db:generate
- **Test ที่ import server-only** ต้องใช้ --conditions=react-server ใช้ npm test หรือคำสั่งในหัวข้อ 5
- **db:seed และ db:reset ไม่ตรวจสอบสภาพแวดล้อม** คำสั่งทั้งสองทำงานกับฐานข้อมูลใดก็ตามที่ DATABASE_URL ระบุ db/clear.ts ปฏิเสธที่จะรันเมื่อ INFISICAL_ENV ตั้งเป็นค่าอื่นที่ไม่ใช่ dev และ db/simple-workflow.ts รันเฉพาะเมื่อเป็น dev
- **ข้อความที่ไม่ถูกต้อง** ข้อความแสดงข้อผิดพลาดสำหรับนักศึกษาเมื่อจำนวนงวดชำระไม่ถูกต้องเขียนว่า "1-4 งวด" (lib/student-error-mapper.ts) แต่ server และฐานข้อมูลรับค่า 1 ถึง 3
- **ไฟล์ที่หลงอยู่ใน Git** test-cases/.~lock.loan-business-logic-test-cases.xlsx# เป็นไฟล์ lock ของ LibreOffice ที่ถูก commit ไปโดยไม่ตั้งใจ ให้ commit การลบไฟล์นี้ และเพิ่ม .~lock.*# ลงใน .gitignore

- **การตั้งค่าที่บันทึกไว้ในเบราว์เซอร์เท่านั้น** หน้าจอข้อมูลติดต่อของผู้ดูแลระบบสูงสุด (SuperAdmin) เก็บเวลาทำการ หมายเหตุวันปิดทำการ และที่อยู่ของคณะไว้ใน `localStorage (metang-system-address)` ไม่ใช่ใน `system_setting` ผู้ใช้คนอื่นจึงไม่เห็นค่าเหล่านี้ เพราะหน้าจอไม่เคยส่งค่าไปยัง server
- **หน้าจอผู้ใช้และบทบาท** (`components/superadmin/setting/UserRolesTab.tsx`) หน้าจอนี้ถือว่า 409 ทุกกรณีคือข้อผิดพลาด "ผู้ดูแลระบบสูงสุด (SuperAdmin) คนสุดท้าย" ในแถวของ ผู้บริหาร และเมื่อ SuperAdmin เปลี่ยนแถวของตนเอง หน้าจอจะมอบบทบาทใหม่ก่อน แล้วลบบทบาทเดิม ไม่สำเร็จ ผู้ใช้จึงอาจได้ทั้งสองบทบาท คู่มือการดูแลรักษาระบบ หัวข้อ 3.2 บอกผู้ปฏิบัติงานว่า ต้องลบบทบาทที่เกิดขึ้นอย่างไร
- **ข้อจำกัดที่ทราบของเวอร์ชัน 0.1.0** คู่มือการดูแลรักษาระบบ หัวข้อ 2.5 ระบุไว้

10. ดูเอกสารให้เป็นปัจจุบัน

ผู้ใช้งานจะได้รับซอร์สโค้ดและ schema ของฐานข้อมูล ดังนั้นเอกสารที่ผิดถือเป็นข้อบกพร่อง ใช้ตารางนี้เมื่อแก้ไขโค้ด

สิ่งที่แก้ไข	สิ่งที่ต้องอัปเดต
<code>db/schema.prisma</code> หรือเพิ่ม migration	<code>db/schema.dbml</code> (ทำด้วยมือ); คู่มือการดูแลรักษาระบบ: จำนวน migration และชื่อล่าสุด (หัวข้อ 1.3, 5.4, 6.4) และรายการ migration ที่ย้อนกลับไม่ได้ (หัวข้อ 6.3); <code>images/developer-guide/er-diagram.mmd</code> หากตารางหรือ key เปลี่ยน; หัวข้อ 6.2 ของคู่มือนี้ หากกฎเปลี่ยน; <code>bruno/README.md</code> และ <code>docs/BRUNO-API-TESTING.md</code> หากอ้างอิงจำนวนดังกล่าว
<code>db/design/database_schema.pdf</code>	เป็นไฟล์เวอร์ชัน 1.0 ลงวันที่ 2026-07-23 ให้สร้างไฟล์ใหม่จาก <code>db/schema.dbml</code> ที่ dbdiagram.io ก่อนมอบให้ผู้ใด
API route	JSDoc tag แล้วรัน <code>npm run openapi:generate</code> ; ไฟล์ request ของ Bruno; จำนวน route และ handler ใน <code>bruno/README.md</code>
จำนวน test	จำนวน test ในคู่มือการดูแลรักษาระบบ หัวข้อ 6.2 ขึ้นตอนที่ 3 และ <code>bruno/README.md</code>
ตัวแปรสภาพแวดล้อม (environment variable)	<code>.env.example</code> และตารางในคู่มือการดูแลรักษาระบบ หัวข้อ 7.1
job หรือความถี่ของ job	<code>lib/jobs/start-scheduler.ts</code> ตาราง job และตัวอย่าง cron ในคู่มือการดูแลรักษาระบบ หัวข้อ 2.3
ข้อความบนหน้าจอ ปุ่ม หรือป้ายสถานะ	คู่มือผู้ใช้ (เจ้าหน้าที่และนักศึกษา) และภาพหน้าจอของคู่มือเหล่านั้น
ขั้นตอนสำหรับผู้ปฏิบัติงาน	คู่มือการดูแลรักษาระบบ และตารางประวัติเอกสารของคู่มือนั้น
สถานะของคำร้อง หรือผู้ที่ยกเลิกได้	หัวข้อ 7.5 ของคู่มือนี้และแผนภาพของหัวข้อนั้น และคู่มือการดูแลรักษาระบบ หัวข้อ 3.2 (แสดงแผนภาพเดียวกัน)
วิธีการทำงานของ outbox	หัวข้อ 8 ของคู่มือนี้และแผนภาพของหัวข้อนั้น และคู่มือการดูแลรักษาระบบ หัวข้อ 2.3 (แสดงแผนภาพเดียวกัน)
แผนภาพ	แก้ไขไฟล์ <code>.mmd</code> แล้ว render ใหม่ (ดูด้านล่าง) และ commit ทั้งสองไฟล์ คู่มือการดูแลรักษาระบบแสดง <code>loan-status-map</code> และ <code>notification-flow</code> จากไฟลเดอร์นี้
คำสั่งหรือกฎสำหรับนักพัฒนา	คู่มือนี้และ <code>AGENTS.md</code>
คู่มือทุกเล่ม (<code>*.md</code> ใน <code>docs/documentation/</code>)	ฉบับอีกภาษาหนึ่งของคู่มือนี้ (คู่มือนักพัฒนาและคู่มือการดูแลรักษาระบบมีฉบับภาษาไทย คือ <code>*.th.md</code>) จากนั้นสร้างไฟล์ PDF ใหม่ (ดูด้านล่าง)

ตรวจสอบจำนวนด้วยคำสั่งเหล่านี้:

```
ls db/migrations | grep -c . # one more than the number of migrations
(migration_lock.toml)
```

```
npm test 2>&1 | grep -E '^i (tests|pass|fail)'
```

```
npm run openapi:generate && git diff --stat public/openapi.json # expect no difference
```

หากต้อง render แผนภาพใหม่ ให้รับคำสั่งนี้ใน docs/documentation/images/developer-guide/ (ต้องมี Node.js และเบราว์เซอร์ Chromium การรันครั้งแรกจะดาวน์โหลด @mermaid-js/mermaid-cli):

```
npx -y -p @mermaid-js/mermaid-cli mmdc -i code-map.mmd -o code-map.png -s 2 -b white -C fonts.css
```

fonts.css บังคับให้ใช้ฟอนต์ Liberation Sans หากไม่มีไฟล์นี้ ป้ายกำกับอาจใช้ฟอนต์อื่น และกล่องอาจตัดข้อความ หาก mmdc หา Chromium ไม่พบ ให้ส่ง -p puppeteer.json ซึ่งเป็นไฟล์ที่มี { "executablePath": "<path to Chromium>", "args": ["--no-sandbox"] }

วิธีสร้างไฟล์ PDF ของคู่มือ (คู่มือนี้และคู่มือการดูแลรักษาระบบทั้งภาษาอังกฤษและภาษาไทย และคู่มือผู้ใช้สองเล่ม):

```
cd docs/docs-pdf
```

```
npm install # once
```

```
npm run render # writes docs/documentation/pdf/*.pdf
```

```
npm test # lays out a test document and the four guides, and checks the PDF files
```

เครื่องมือนี้ต้องใช้ Chromium (CHROMIUM_PATH ค่าเริ่มต้นคือ /usr/bin/chromium), pdftotext, pdffonts และ pdfimages จาก poppler-utils และฟอนต์ Liberation Sans, Liberation Mono และ Noto Sans Thai เครื่องมือนี้แยกจากแอปพลิเคชัน: npm ci ที่ root ไม่ติดตั้งให้, npm test ของแอปพลิเคชันไม่รัน และ CI ไม่ใช้

ก่อนพิมพ์แต่ละหน้า เครื่องมือจะวัด layout npm run render จะหยุดด้วย exit code 1 และระบุหัวข้อ เมื่อคอลัมน์ของตาราง ถูกบีบ คำถูกตัดกลางคำ ข้อความล้นขอบหน้าหรือถูกตัดขาด หรือรูปภาพโหลดไม่ได้ การทดสอบยังตรวจ PDF ที่พิมพ์ออกมา ด้วย: ข้อความอยู่ภายในขอบกระดาษ ไม่มีหน้าว่าง ไม่มีตัวอักษรหาย ใช้เฉพาะฟอนต์ที่คาดไว้ และทุกตัวอักษรในคู่มือมี glyph หากพบข้อความชนิดใหม่ที่พิมพ์ออกมาไม่ดี ให้เพิ่มเป็นหัวข้อใน docs/docs-pdf/test/fixture.mjs ก่อน แล้วจึงแก้ไข render.mjs